Firmware for Position Sense He3 detector

INTRODUCTION

This document explain the usage of the PSA firmware using one of the supported CAEN hardware platform to measure interaction position of high energy particles (i.e. neutrons) on position sense detector like He3 proportional position sense tube. This detector has a resistive electrode. The electrode is read on both side. The relative amplitude of the signal generated on the two side is proportional to the position of the interaction. If the particle interact in the middle of the tube the signal generated has the same amplitude on both side of the detector while if the particle hits the tube closer to one side the two signal are not the same.

The position of interaction can be calculated as

$$Position = \frac{Q_a}{Q_a + Q_b} \ \alpha \ \frac{E_a}{E_a + E_b}$$

Where Q is the charge collected from the preamplifier on both side indeed proportional to the energy

On each channel the firmware uses a digital shaper to calculate the energy and provide both energy on the output.

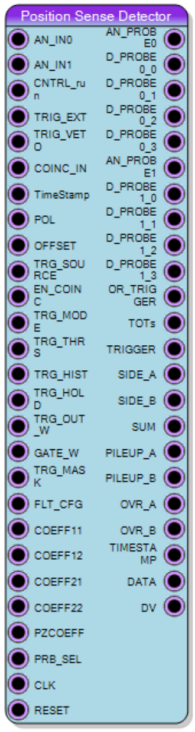
The output packet generated by the firmware contains the timestamp, the energy on side A and energy on side B. The user on the PC is than able to reconstruct both the energy spectrum (as Ea + Eb), the time distribution (using the absolute timestamp) and the position (calculating the ratio between Ea and the E)

SUPPORTED HARDWARE PLATFORM

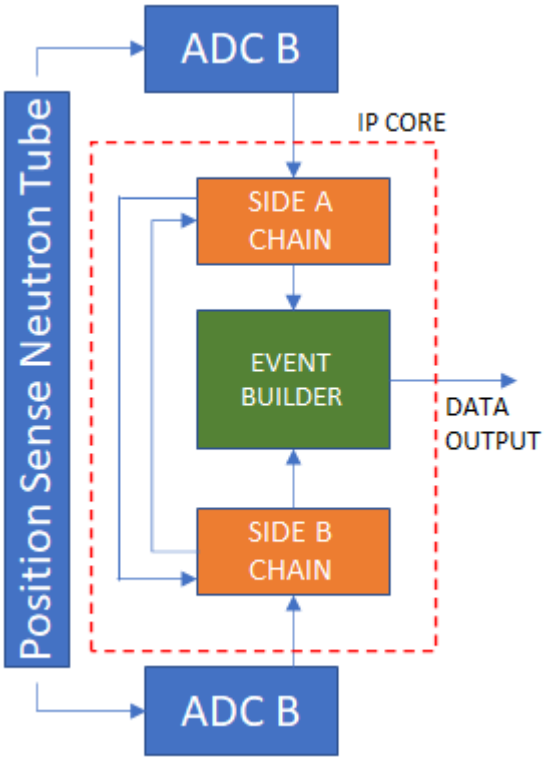
Model	**Detectors (pair) per section**	**Total Detectors**
R5560/A	16	64
R5560/B	16	64
DT5560	16	16
R5560SE/A	16	64
R5560SE/B	16	64
DT1260	1	1

PSA CORE

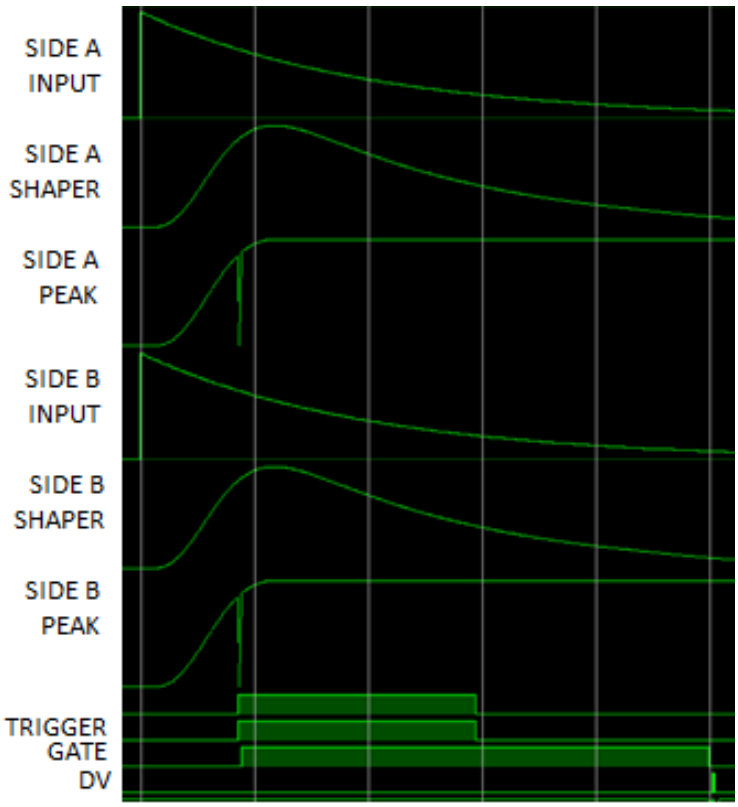
The core of the PSA is a single channel module that receive in input the two analog signals, each from one side of the detector, and include trigger, baseline restored, digital shaper, peak detector and holder. The core is than replicated for the number of detector supported by the hardware platform



The IP integrates two data processing circuits for side A and side B of the detector and use the information from both side to trigger and calculate the energy A/B and energy sum. SIDE A/B emulate a traditional quasi gaussian shaper followed by a peak holder

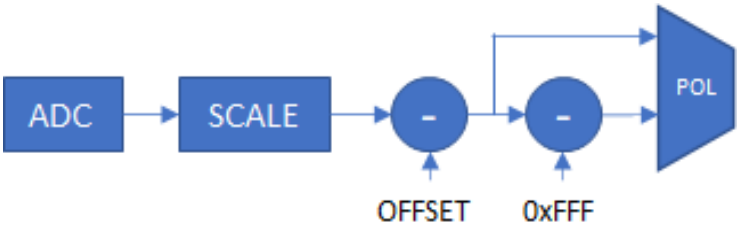


Example of signal shape on both side of detector



Input stage

The filter implements the Gaussian shaping filter, and accept current amplifier signals, or charge amplifier signals. It include a pole-zero compensation, trigger circuit and data packet generator.
The core works on 12 bit data indeed, selecting the correct board ADC number of bits, the core will automatically scales the input numbers in order to fit to the 12 bit data format
A first input block allows to subtract the offset on the eventually invert the polarity (POL=0 positive)



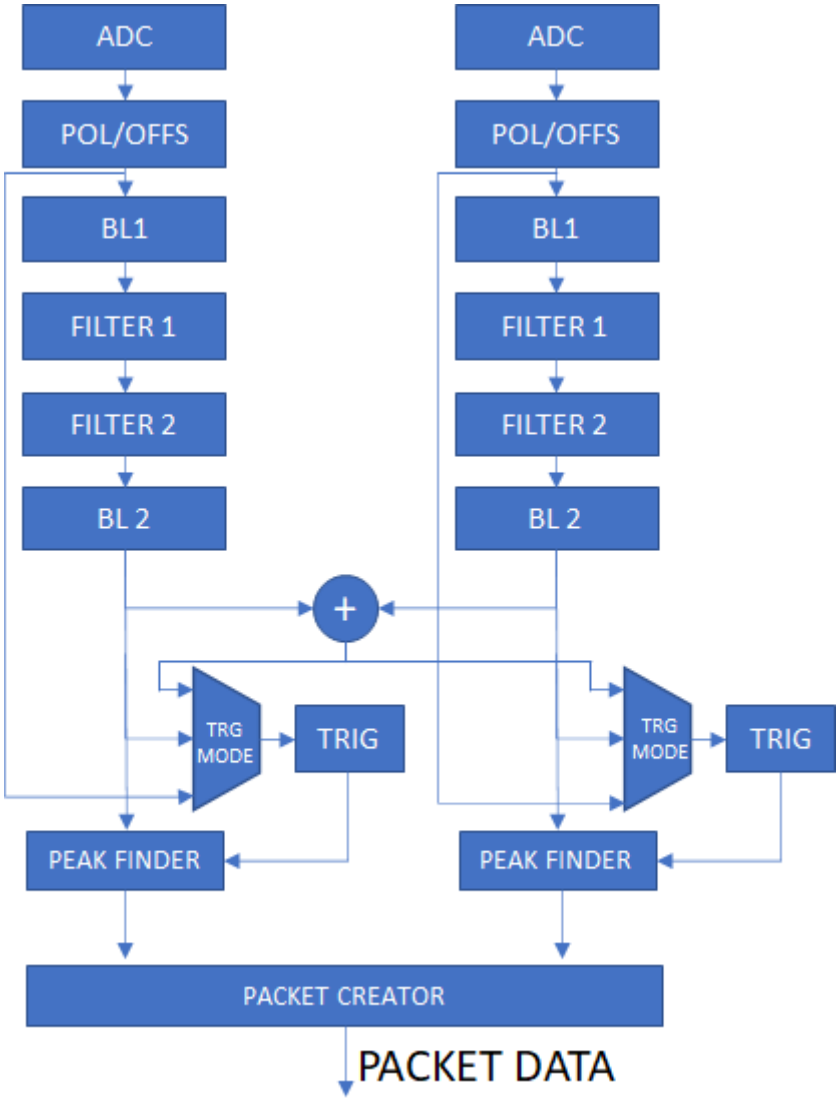
Signal filter

The channel signal treatment block (main filter) consists of 5 elements:

1. A first baseline correction circuit, applied to the incoming signal
2. A pole-zero compensation circuit
3. A first second-order gaussian filter section
4. A second second-order gaussian filter section
5. A final baseline correction circuit

The input is 12 bits unsigned data, the output is 16 bits unsigned data. Each of the 5 blocs can be activated or de-activated (in which case it just transmits the input to the output).
In between blocs, the signals are of the signed integer type, so that positive as well as negative samples can be treated. At the output, the negative values are clipped to 0, to be able to put out 16 bits of unsigned positive values.

All blocs work with fixed-point arithmetic. In fact, they work with integer arithmetic, but the integer values are left-shifted a certain number of bits in order to be able to work with a certain binary fraction, in order not to accumulate too many significant rounding errors.
The filter is applied continuously to the incoming data stream from the ADC for every channel independently. There is a latency of a few clock ticks and the resulting output signal is continuously put out.



Trigger

The block integrate also the trigger logic. Trigger can be configured to operate on the output of the signal filter or on the input signal after the polarity/offset regulation.

Further more it is possible to select the trigger soruce between:

- sum of the signal from both side (A/B)
- external only
- or of two side A/B
- and of two side A/B

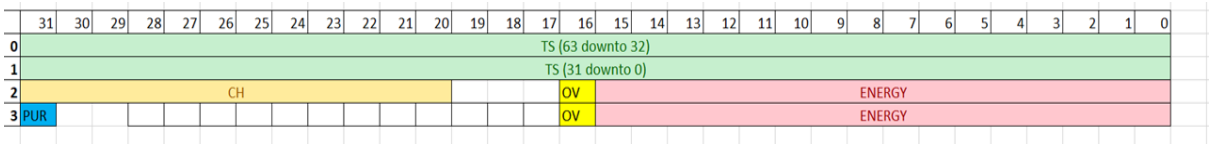
Maximum finder

When the trigger is fired (ie when the sum of the two paired channel samples crosses the channel threshold value), a maximum finder on the sum signal will find that sample which is the largest since the trigger was fired. A programmable length gate window is open in order to find the absolute maximum.
The corresponding local channel sample value is stored until a larger value on the sum is found until the peak gate window closes. At the end of the gate window the events from both side of the detector chain are merged in a single packed and the output is generated.
If a new trigger is issued before the closure of the gate window the event is marked as piled-up and the second one is discarded.
The local output sample value corresponding to the time tick when the sum of the two channel outputs reached its global maximum since the threshold crossing.

Output data formatter

The trigger fires simultaneously on both side A and side B of the processing chain. At the end of the gate window the two processing chains provide in output the energy, the timestamp, the flags (pileup/over-range). This information are made available on the output as separate bus and as a standard packet that can be transfered directly to the user software using a custom packet or a list. the DV signal indicates when the data packet is available to the user

Following data format is used, where the bit col 31, row 0 is the MSB (bit 127) and it col 0 row 3 is the LSB (bit 0) in the data packet



If validation required register is enabled, the readout discard the packet if the coinc_i is not high for at least one clock cycles during the gate window.

Preamp pole zero

The continuous-time domain pole zero compensation comes down in accepting a signal that went through a first order system with a long time constant, and modifying this such that it looks as if the signal went trough a first order system with a much shorter time constant. This can simply be implemented with the following transfer function:

$$H(s)=\frac{s+1/\tau_l}{s+1/\tau_s}$$

where tau-s is the short time constant and tau-l is the to be compensated long

time constant. Here, tau-s is our own choice, but tau-l has to be precisely matched to the time constant of the system of which the pole has to be compensated.

We can re-write our function as:

$$H(s)=\frac{s+1/\tau_l}{s+1/\tau_s}=1+\frac{1/\tau_l-1/\tau_s}{s+1/\tau_s}$$

This is now in the form onto which we can apply the impulse invariance transformation in the discrete domain:

$$Z(z)=1+(T_s/\tau_l-T_s/\tau_s)\frac{1}{1-e^{-T_s/\tau_s}z^{-1}}$$

Now, in as much as tau-s is several times the sample period (we will take this equal to 8 times) we can approximate

$$e^{-T_s/\tau_s}\approx 1-\frac{T_s}{\tau_s}$$

As such, this can be implemented as a linear combination of the direct input, and a first order IIR filter:

$$w[n]=i[n]+(1-\frac{T_s}{\tau_s})w[n-1] \quad \text{and}$$

$$u[n]=i[n]+(T_s/\tau_l-T_s/\tau_s)w[n]$$

where again, we calculate in fixed point integers, and "shift" everything with a certain power of 2 up to allow integer multiplication, followed by a division (and truncation) by the same power of 2. We used 2 to the power 15 for the coefficient Ts/tau_l (and also Ts/tau_s which is a fixed choice

Gaussian Filter

The analogue Gaussian filter is a 4th order approximation of a true Gaussian profile.

The normalized transfer function looks like:

$$H(s)=\frac{4.899}{4.899+11.42\,s+10.87\,s^2+5.073\,s^3+s^4}$$

This filter has 4 poles in the complex plane:

- -1.35536 +/- I 0.32795
- -1.18108 +/- I 1.0604

This is the normalized filter for our "time constant" tau equal to 2 Pi. For a time constant "tau" in general, we have to divide these poles by tau times 2 Pi.

There are different ways of transforming a continuous-time system transfer function into the discrete time domain.

If we apply the impulse invariance method, which is based upon the preservation of the discrete samples of the impulse response of the continuous-time domain as the samples

of the discrete-time impulse response, we can only transform exponential components, that is to say, components of the form:

$H(s)=\frac{1}{s-s_0}$ which have a continuous-time impulse response $h(t)=e^{s_0 t}$

We can then sample this continuous-time impulse response at sample period Ts:

$h_Z(n)=e^{s_0 n T_s}=T_s h(n T_s)=T_s (e^{s T_s})^n$ which is a geometric series, which can be represented in the

Z-domain as:

$$H_Z(z)=\frac{T_s}{1-e^{s_0 T_s} z^{-1}}$$

If our continuous-time transfer function can be written as a linear combination of this kind of single-pole functions, the technique generalizes:

$H(s)=\sum_i \frac{A_i}{s-s_i}$ gives $h(t)=\sum_i A_i e^{s_i t}$ so $h_Z(n)=T_s h(nT_s)=T_s \sum_i A_i e^{s_0 nT_s}$

The corresponding Z-transform then comes down to:

$$H_Z(z)=T_s \sum_i \frac{A_i}{1-e^{s_i T_s} z^{-1}}$$

As such, we see that if we can write the original continuous-time domain transfer function as a partial-fraction expansion with simple terms (which means that we don't have double poles), we can obtain the partial-fraction expansion of the corresponding Z-transform with the same coefficients (Ai in our formula), and with the poles undergoing the following mapping: $s_i \rightarrow e^{s_i T_s}$.

However, note that in principle, the recombination of all these terms into a single rational function does imply the possible appearance of finite zeros.

We will neglect these zeros, and assume these zeros to remain at the origin.

In fact, this comes down of applying, not the impulse invariance method, but rather the “matched Z-transform method”. Yet another method is the bilinear transformation.

In the matched Z-transform method, poles *and* zeros are mapped with the relationship $s_i \rightarrow e^{s_i T_s}$.

The reason why we want to neglect the finite zeros is that if we do not do so, we will need twice as many multiplications in the filter implementation, than when we assume the zeros at 0, and we don't have enough computing resources to do so in the FPGA. The zeros are much less important than the poles, as the zeros apply a FIR filter to the input of a length equal to the order of the filter (which will be 4). That would only have a significant impact[1] if the sum of the numerator coefficients were 0, which isn't the case. In fact, it will turn out that the coefficients of the numerator will be all positive, in which case this FIR filter will only calculate a kind of weighted average over 5 successive samples. It will moreover turn out that only 3 coefficients have any significant value, so the numerator of the filter will come down to a weighted moving average function of the input over 3 samples. This can be neglected if the time constant of the filter is much more than 3 samples, which it will be. The denominator (which determines the IIR filter) will by far have the dominant effect on the impulse response, which is why we limit ourselves to the implementation of the denominator.

As such, we simply obtain that we have to use the following expression as our Z-transform transfer function:

$$H_Z(z) = \frac{constant}{\prod_i 1 - e^{s_i T_s / (2\pi\tau)} Z^{-1}}$$ where I runs over the 4 s-plane poles mentioned earlier.

1 Indeed, in that case, the FIR filter would implement a kind of derivative.

This will result in two second-order factors for the denominator:

$(1-a_1 z^{-1}+b_1 z^{-2})(1-a_2 z^{-1}+b_2 z^{-2})$ which can then easily be implemented as a succession of two second-order IIR filters.

The coefficients can be calculated as follows, expanding the two factors (corresponding to the two conjugate poles each time):

$$a_1 = 2e^{-1.355T}\cos(0.3279T)$$

$$b_1 = e^{-2.711T}$$

$$a_2 = 2e^{-1.181T}\cos(1.0604T)$$

$$b_2 = e^{-2.362T}$$

where $T = \frac{I_s}{2\pi\tau}$ with Ts the sample period of the digitizer (62.5 MHz) and tau the time constant

of the Gaussian filter. Note the relationship to the original poles: the a coefficient is twice the exponent of the real part times the cosine of the imaginary part, while the b coefficient is the exponent of twice the real part. This is obvious if one remembers that it comes from

$$(1-e^{-s_1 T} z^{-1})(1-(e^{-s_1 T})^{*} z^{-1})$$

The IIR filter section of second order is then simply implemented as the following iteration:

$$u[n]=i[n]+a_k u[n-1]-b_k u[n-2]$$

The fixed-point implementation will consist in multiplying the constants a1 and b1 with a power of 2, applying the (integer) calculation, and dividing the result by the same power of 2. We opted for the 14^{th} power of 2. It can also be interesting to multiply input and output with a (small) power of two to get some "digits after the comma". These can be removed at the end of the calculation, to take care of part of the rounding errors.

Baseline correction

The baseline correction does two things: it neutralizes offsets in the input signal, and it adjusts the signal level "in between pulses" to 0. This means that the average of the signal is not 0 (given that the pulses are positive), and hence that the AC coupling which is supposed to be somewhere along the signal chain (for instance, the HV blocking capacitor) is undone. The baseline correction is a non-linear operation.

The proposed method is the following: the signal samples are chopped up in chunks of length N. In every chunk of length N, the minimal sample value is determined. So every N samples, a new minimal sample value is found. This minimum is then fed into a first order low pass digital filter:

$$w[n]=\frac{1}{k} i[n]+(1-\frac{1}{k})w[n-1]$$

If we take k to be a power of 2, this can be implemented easily with just shifting binary words. We opted for k = 32. Note that i[n] is the minimum of the latest chunk, and that n counts the number of chunks. w[n] is then our estimation of the baseline level in the input signal.

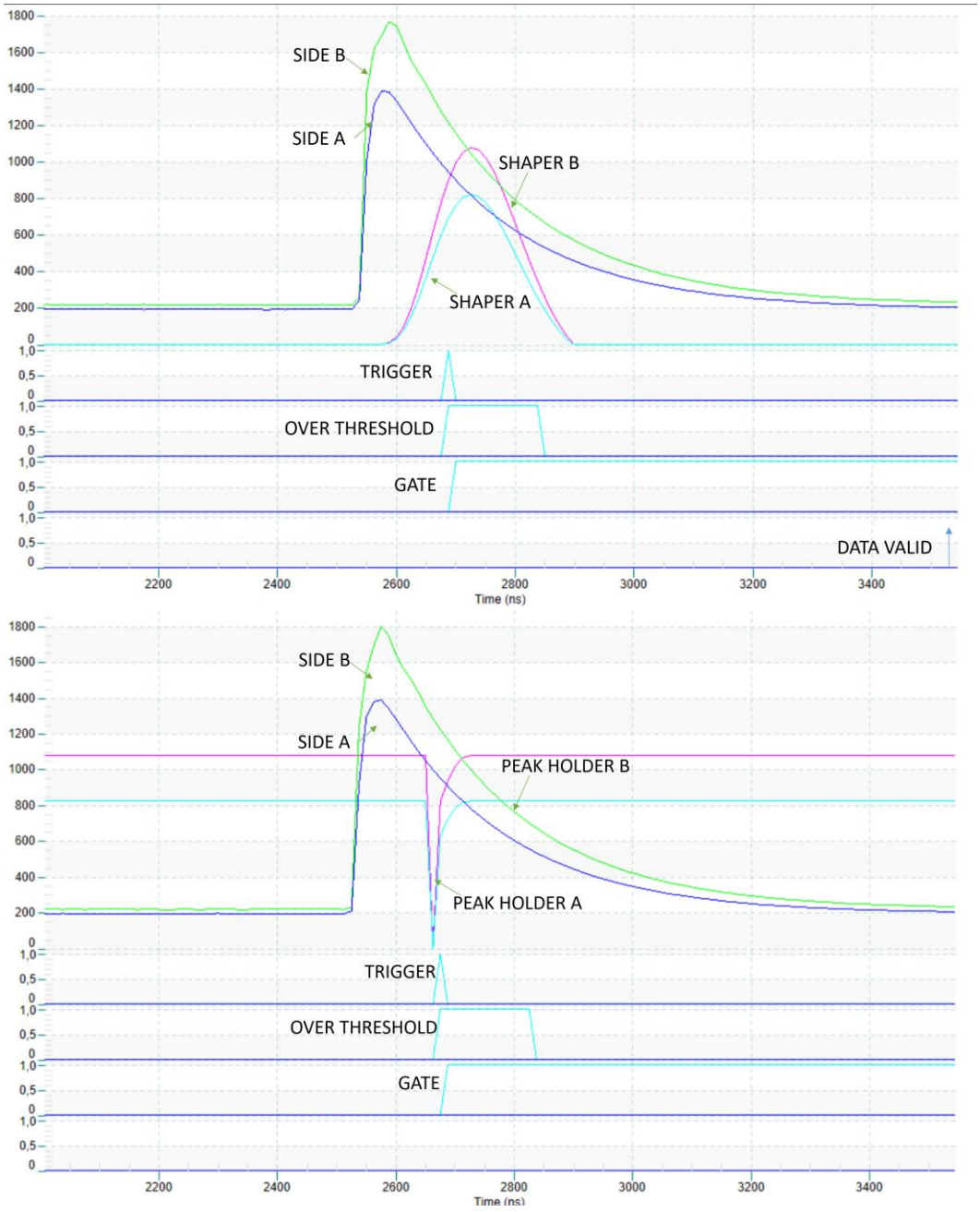
The baseline-corrected output signal is then simply: $u[N*n+m]=i[N*n+m]-w[n];m<N$

:

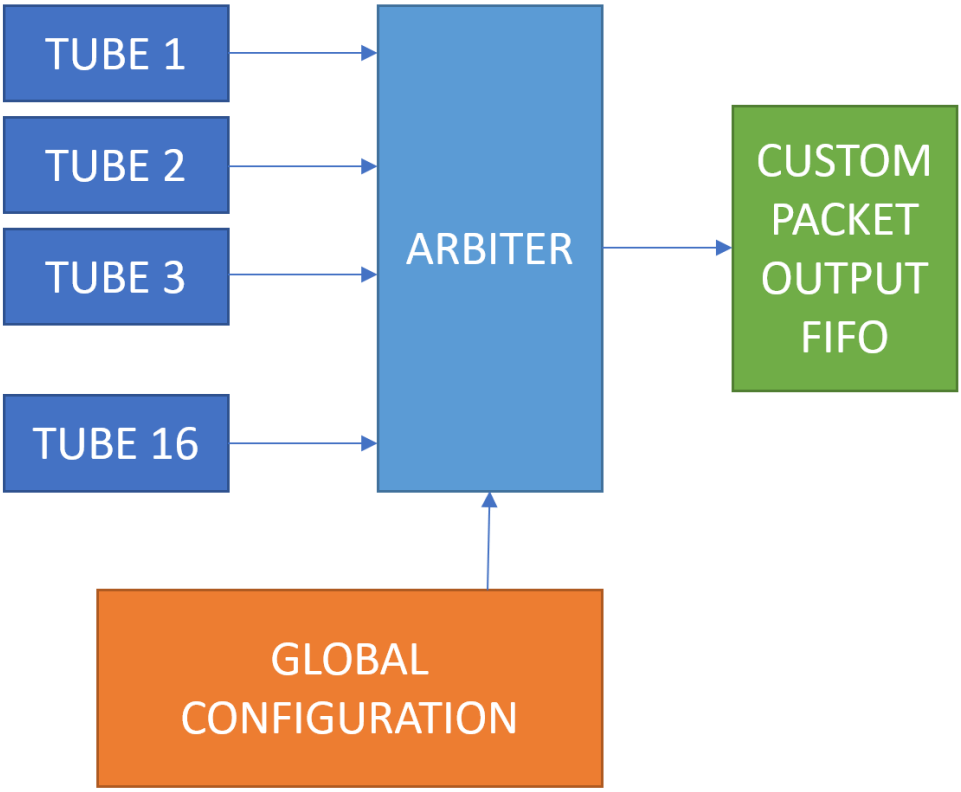
during the next N samples we correct with the latest baseline estimator.

This baseline correction has of course a small error in it: the minimum will also take the minimum of the noise excursions instead of the average baseline value. In other words, on a signal with only noise and no pulses, the average level after correction will not be 0, but it will be the negative of the average negative excursion (due to noise) on N samples, which amounts to a few sigma of the noise level if the noise has a Gaussian distribution.

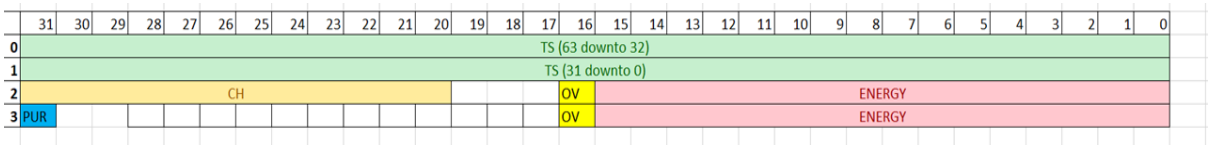
PSA SIGNALS



FULL FIRMWARE STRUCTURE (ONLY FOR R/DT5560/SE)



Multiple PSA core (TUBE x) produce in output a packet for each event



The event is than multiplexed by an arbiter. The arbiter has inside 128 elements FIFO for each 16 channels and mux the channel on a single output

The output of the arbiter is than forwarded to the custom packet block where an header of 32 bit (0xFFFFFFFF) is put on the front of the PSA output.

The global configuration is a group of register (described later in the document) to configure the IP.

On side of this main datapath an oscilloscope and a spectrum block is used as monitor

REGISTERS AND READOUT BLOCKS

NAME	TYPE	ADDRESS	FUNCTION
T0SW	REGISTER	0x11	Software reset of T0
VETOSW	REGISTER	0x12	Software veto
EXTTRG_EN	REGISTER	0x13	Enable external trigger
RUN_EN	REGISTER	0x14	Enable filter processing
RESET_CORE	REGISTER	0x15	Reset internal filter core
PCFG_RAWOFS	REGISTER	0x1	Offset on 14/16 bit input signal
PCFG_POL	REGISTER	0x2	Polarity
PCFG_TRG_SOURCE	REGISTER	0x3	Trigger source
PCFG_TRG_MODE	REGISTER	0x4	Trigger mode
PCFG_THRS	REGISTER	0x5	Trigger threshold
PCFG_HIST	REGISTER	0x6	Trigger hysteresis
PCFG_TRG_HOLD	REGISTER	0x7	Trigger hold off
PCFG_TRG_W	REGISTER	0x8	Trigger pulse width
PCFG_GATE_W	REGISTER	0x9	Peak searcher window length
PCFG_FLT_CFG	REGISTER	0xa	Filter configuration bits
PCFG_C11	REGISTER	0xb	Coefficient 11
PCFG_C12	REGISTER	0xc	Coefficient 12
PCFG_C21	REGISTER	0xd	Coefficient 21
PCFG_C22	REGISTER	0xe	Coefficient 22
PCFG_CPZ	REGISTER	0xf	Coefficient PZ
PCFG_PRB_SEL	REGISTER	0x10	Analog probe selector
TRG_MASK_CH0	REGISTER	0x20002	Trigger Mask CH0
TRG_MASK_CH1	REGISTER	0x20003	Trigger Mask CH1
TRG_MASK_CH2	REGISTER	0x20004	Trigger Mask CH2
TRG_MASK_CH3	REGISTER	0x20005	Trigger Mask CH3
TRG_MASK_CH4	REGISTER	0x20006	Trigger Mask CH4
TRG_MASK_CH5	REGISTER	0x20007	Trigger Mask CH5
TRG_MASK_CH6	REGISTER	0x20008	Trigger Mask CH6
TRG_MASK_CH7	REGISTER	0x20009	Trigger Mask CH7
TRG_MASK_CH8	REGISTER	0x2000a	Trigger Mask CH8
TRG_MASK_CH9	REGISTER	0x2000b	Trigger Mask CH9
TRG_MASK_CH10	REGISTER	0x2000c	Trigger Mask CH10
TRG_MASK_CH11	REGISTER	0x2000d	Trigger Mask CH11
TRG_MASK_CH12	REGISTER	0x2000e	Trigger Mask CH12
TRG_MASK_CH13	REGISTER	0x2000f	Trigger Mask CH13
TRG_MASK_CH14	REGISTER	0x20010	Trigger Mask CH14
TRG_MASK_CH15	REGISTER	0x20011	Trigger Mask CH15
FINE_OFS_CH0	REGISTER	0x20014	Fine offset (on 12 bit) CH0
FINE_OFS_CH1	REGISTER	0x20015	Fine offset (on 12 bit) CH1
FINE_OFS_CH2	REGISTER	0x20016	Fine offset (on 12 bit) CH2
FINE_OFS_CH3	REGISTER	0x20017	Fine offset (on 12 bit) CH3

FINE_OFS_CH4	REGISTER	0x20018	Fine offset (on 12 bit) CH4
FINE_OFS_CH5	REGISTER	0x20019	Fine offset (on 12 bit) CH5
FINE_OFS_CH6	REGISTER	0x2001a	Fine offset (on 12 bit) CH6
FINE_OFS_CH7	REGISTER	0x2001b	Fine offset (on 12 bit) CH7
FINE_OFS_CH8	REGISTER	0x2001c	Fine offset (on 12 bit) CH8
FINE_OFS_CH9	REGISTER	0x2001d	Fine offset (on 12 bit) CH9
FINE_OFS_CH10	REGISTER	0x2001e	Fine offset (on 12 bit) CH10
FINE_OFS_CH11	REGISTER	0x2001f	Fine offset (on 12 bit) CH11
FINE_OFS_CH12	REGISTER	0x20020	Fine offset (on 12 bit) CH12
FINE_OFS_CH13	REGISTER	0x20021	Fine offset (on 12 bit) CH13
FINE_OFS_CH14	REGISTER	0x20022	Fine offset (on 12 bit) CH14
FINE_OFS_CH15	REGISTER	0x20023	Fine offset (on 12 bit) CH15
RateMeter_0	MCRateMeter	0x10000	Rate meter base address
Oscilloscope_0	Oscilloscope	0x21000	Oscilloscope base address
Oscilloscope_0_READ_STATUS	CNTRL-REGISTER	0x22000	Oscilloscope read status
Oscilloscope_0_READ_POSITION	CNTRL-REGISTER	0x22001	Oscilloscope read position
Oscilloscope_0_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x22002	Oscilloscope config trigger mode
Oscilloscope_0_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x22003	Oscilloscope config pretrigger
Oscilloscope_0_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x22004	Oscilloscope config trigger level
Oscilloscope_0_CONFIG_ARM	CNTRL-REGISTER	0x22005	Oscilloscope config arm
Oscilloscope_0_CONFIG_DECIMATOR	CNTRL-REGISTER	0x22006	Oscilloscope config decimator
Oscilloscope_1	Oscilloscope	0x23000	Oscilloscope base address
Oscilloscope_1_READ_STATUS	CNTRL-REGISTER	0x24000	Oscilloscope read status
Oscilloscope_1_READ_POSITION	CNTRL-REGISTER	0x24001	Oscilloscope read position
Oscilloscope_1_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x24002	Oscilloscope config trigger mode
Oscilloscope_1_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x24003	Oscilloscope config pretrigger
Oscilloscope_1_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x24004	Oscilloscope config trigger level
Oscilloscope_1_CONFIG_ARM	CNTRL-REGISTER	0x24005	Oscilloscope config arm
Oscilloscope_1_CONFIG_DECIMATOR	CNTRL-REGISTER	0x24006	Oscilloscope config decimator
Oscilloscope_2	Oscilloscope	0x25000	Oscilloscope base address
Oscilloscope_2_READ_STATUS	CNTRL-REGISTER	0x26000	Oscilloscope read status
Oscilloscope_2_READ_POSITION	CNTRL-REGISTER	0x26001	Oscilloscope read position
Oscilloscope_2_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x26002	Oscilloscope config trigger mode
Oscilloscope_2_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x26003	Oscilloscope config pretrigger
Oscilloscope_2_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x26004	Oscilloscope config trigger level
Oscilloscope_2_CONFIG_ARM	CNTRL-REGISTER	0x26005	Oscilloscope config arm
Oscilloscope_2_CONFIG_DECIMATOR	CNTRL-REGISTER	0x26006	Oscilloscope config decimator
Oscilloscope_4	Oscilloscope	0x27000	Oscilloscope base address
Oscilloscope_4_READ_STATUS	CNTRL-REGISTER	0x28000	Oscilloscope read status
Oscilloscope_4_READ_POSITION	CNTRL-REGISTER	0x28001	Oscilloscope read position
Oscilloscope_4_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x28002	Oscilloscope config trigger mode
Oscilloscope_4_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x28003	Oscilloscope config pretrigger
Oscilloscope_4_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x28004	Oscilloscope config trigger level
Oscilloscope_4_CONFIG_ARM	CNTRL-REGISTER	0x28005	Oscilloscope config arm

Oscilloscope_4_CONFIG_DECIMATOR	CNTRL-REGISTER	0x28006	Oscilloscope config decimator
Oscilloscope_3	Oscilloscope	0x29000	Oscilloscope base address
Oscilloscope_3_READ_STATUS	CNTRL-REGISTER	0x2a000	Oscilloscope read status
Oscilloscope_3_READ_POSITION	CNTRL-REGISTER	0x2a001	Oscilloscope read position
Oscilloscope_3_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x2a002	Oscilloscope config trigger mode
Oscilloscope_3_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x2a003	Oscilloscope config pretrigger
Oscilloscope_3_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x2a004	Oscilloscope config trigger level
Oscilloscope_3_CONFIG_ARM	CNTRL-REGISTER	0x2a005	Oscilloscope config arm
Oscilloscope_3_CONFIG_DECIMATOR	CNTRL-REGISTER	0x2a006	Oscilloscope config decimator
Oscilloscope_5	Oscilloscope	0x2b000	Oscilloscope base address
Oscilloscope_5_READ_STATUS	CNTRL-REGISTER	0x2c000	Oscilloscope read status
Oscilloscope_5_READ_POSITION	CNTRL-REGISTER	0x2c001	Oscilloscope read position
Oscilloscope_5_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x2c002	Oscilloscope config trigger mode
Oscilloscope_5_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x2c003	Oscilloscope config pretrigger
Oscilloscope_5_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x2c004	Oscilloscope config trigger level
Oscilloscope_5_CONFIG_ARM	CNTRL-REGISTER	0x2c005	Oscilloscope config arm
Oscilloscope_5_CONFIG_DECIMATOR	CNTRL-REGISTER	0x2c006	Oscilloscope config decimator
Oscilloscope_8	Oscilloscope	0x2d000	Oscilloscope base address
Oscilloscope_8_READ_STATUS	CNTRL-REGISTER	0x2e000	Oscilloscope read status
Oscilloscope_8_READ_POSITION	CNTRL-REGISTER	0x2e001	Oscilloscope read position
Oscilloscope_8_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x2e002	Oscilloscope config trigger mode
Oscilloscope_8_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x2e003	Oscilloscope config pretrigger
Oscilloscope_8_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x2e004	Oscilloscope config trigger level
Oscilloscope_8_CONFIG_ARM	CNTRL-REGISTER	0x2e005	Oscilloscope config arm
Oscilloscope_8_CONFIG_DECIMATOR	CNTRL-REGISTER	0x2e006	Oscilloscope config decimator
Oscilloscope_9	Oscilloscope	0x2f000	Oscilloscope base address
Oscilloscope_9_READ_STATUS	CNTRL-REGISTER	0x30000	Oscilloscope read status
Oscilloscope_9_READ_POSITION	CNTRL-REGISTER	0x30001	Oscilloscope read position
Oscilloscope_9_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x30002	Oscilloscope config trigger mode
Oscilloscope_9_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x30003	Oscilloscope config pretrigger
Oscilloscope_9_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x30004	Oscilloscope config trigger level
Oscilloscope_9_CONFIG_ARM	CNTRL-REGISTER	0x30005	Oscilloscope config arm
Oscilloscope_9_CONFIG_DECIMATOR	CNTRL-REGISTER	0x30006	Oscilloscope config decimator
Oscilloscope_10	Oscilloscope	0x31000	Oscilloscope base address
Oscilloscope_10_READ_STATUS	CNTRL-REGISTER	0x32000	Oscilloscope read status
Oscilloscope_10_READ_POSITION	CNTRL-REGISTER	0x32001	Oscilloscope read position
Oscilloscope_10_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x32002	Oscilloscope config trigger mode
Oscilloscope_10_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x32003	Oscilloscope config pretrigger
Oscilloscope_10_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x32004	Oscilloscope config trigger level
Oscilloscope_10_CONFIG_ARM	CNTRL-REGISTER	0x32005	Oscilloscope config arm
Oscilloscope_10_CONFIG_DECIMATOR	CNTRL-REGISTER	0x32006	Oscilloscope config decimator
Oscilloscope_11	Oscilloscope	0x33000	Oscilloscope base address
Oscilloscope_11_READ_STATUS	CNTRL-REGISTER	0x34000	Oscilloscope read status
Oscilloscope_11_READ_POSITION	CNTRL-REGISTER	0x34001	Oscilloscope read position

Oscilloscope_11_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x34002	Oscilloscope config trigger mode
Oscilloscope_11_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x34003	Oscilloscope config pretrigger
Oscilloscope_11_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x34004	Oscilloscope config trigger level
Oscilloscope_11_CONFIG_ARM	CNTRL-REGISTER	0x34005	Oscilloscope config arm
Oscilloscope_11_CONFIG_DECIMATOR	CNTRL-REGISTER	0x34006	Oscilloscope config decimator
Oscilloscope_12	Oscilloscope	0x35000	Oscilloscope base address
Oscilloscope_12_READ_STATUS	CNTRL-REGISTER	0x36000	Oscilloscope read status
Oscilloscope_12_READ_POSITION	CNTRL-REGISTER	0x36001	Oscilloscope read position
Oscilloscope_12_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x36002	Oscilloscope config trigger mode
Oscilloscope_12_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x36003	Oscilloscope config pretrigger
Oscilloscope_12_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x36004	Oscilloscope config trigger level
Oscilloscope_12_CONFIG_ARM	CNTRL-REGISTER	0x36005	Oscilloscope config arm
Oscilloscope_12_CONFIG_DECIMATOR	CNTRL-REGISTER	0x36006	Oscilloscope config decimator
Oscilloscope_13	Oscilloscope	0x37000	Oscilloscope base address
Oscilloscope_13_READ_STATUS	CNTRL-REGISTER	0x38000	Oscilloscope read status
Oscilloscope_13_READ_POSITION	CNTRL-REGISTER	0x38001	Oscilloscope read position
Oscilloscope_13_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x38002	Oscilloscope config trigger mode
Oscilloscope_13_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x38003	Oscilloscope config pretrigger
Oscilloscope_13_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x38004	Oscilloscope config trigger level
Oscilloscope_13_CONFIG_ARM	CNTRL-REGISTER	0x38005	Oscilloscope config arm
Oscilloscope_13_CONFIG_DECIMATOR	CNTRL-REGISTER	0x38006	Oscilloscope config decimator
Oscilloscope_6	Oscilloscope	0x39000	Oscilloscope base address
Oscilloscope_6_READ_STATUS	CNTRL-REGISTER	0x3a000	Oscilloscope read status
Oscilloscope_6_READ_POSITION	CNTRL-REGISTER	0x3a001	Oscilloscope read position
Oscilloscope_6_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x3a002	Oscilloscope config trigger mode
Oscilloscope_6_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x3a003	Oscilloscope config pretrigger
Oscilloscope_6_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x3a004	Oscilloscope config trigger level
Oscilloscope_6_CONFIG_ARM	CNTRL-REGISTER	0x3a005	Oscilloscope config arm
Oscilloscope_6_CONFIG_DECIMATOR	CNTRL-REGISTER	0x3a006	Oscilloscope config decimator
Oscilloscope_7	Oscilloscope	0x3b000	Oscilloscope base address
Oscilloscope_7_READ_STATUS	CNTRL-REGISTER	0x3c000	Oscilloscope read status
Oscilloscope_7_READ_POSITION	CNTRL-REGISTER	0x3c001	Oscilloscope read position
Oscilloscope_7_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x3c002	Oscilloscope config trigger mode
Oscilloscope_7_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x3c003	Oscilloscope config pretrigger
Oscilloscope_7_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x3c004	Oscilloscope config trigger level
Oscilloscope_7_CONFIG_ARM	CNTRL-REGISTER	0x3c005	Oscilloscope config arm
Oscilloscope_7_CONFIG_DECIMATOR	CNTRL-REGISTER	0x3c006	Oscilloscope config decimator
Oscilloscope_14	Oscilloscope	0x3d000	Oscilloscope base address
Oscilloscope_14_READ_STATUS	CNTRL-REGISTER	0x3e000	Oscilloscope read status
Oscilloscope_14_READ_POSITION	CNTRL-REGISTER	0x3e001	Oscilloscope read position
Oscilloscope_14_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x3e002	Oscilloscope config trigger mode
Oscilloscope_14_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x3e003	Oscilloscope config pretrigger
Oscilloscope_14_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x3e004	Oscilloscope config trigger level
Oscilloscope_14_CONFIG_ARM	CNTRL-REGISTER	0x3e005	Oscilloscope config arm

Oscilloscope_14_CONFIG_DECIMATOR	CNTRL-REGISTER	0x3e006	Oscilloscope config decimator
Oscilloscope_15	Oscilloscope	0x3f000	Oscilloscope base address
Oscilloscope_15_READ_STATUS	CNTRL-REGISTER	0x40000	Oscilloscope read status
Oscilloscope_15_READ_POSITION	CNTRL-REGISTER	0x40001	Oscilloscope read position
Oscilloscope_15_CONFIG_TRIGGER_MODE	CNTRL-REGISTER	0x40002	Oscilloscope config trigger mode
Oscilloscope_15_CONFIG_PRETRIGGER	CNTRL-REGISTER	0x40003	Oscilloscope config pretrigger
Oscilloscope_15_CONFIG_TRIGGER_LEVEL	CNTRL-REGISTER	0x40004	Oscilloscope config trigger level
Oscilloscope_15_CONFIG_ARM	CNTRL-REGISTER	0x40005	Oscilloscope config arm
Oscilloscope_15_CONFIG_DECIMATOR	CNTRL-REGISTER	0x40006	Oscilloscope config decimator
Spectrum_0	Spectrum	0x50000	Spectrum base address
Spectrum_0_STATUS	CNTRL-REGISTER	0x60000	Spectrum status
Spectrum_0_CONFIG	CNTRL-REGISTER	0x60001	Spectrum config
Spectrum_0_CONFIG_LIMIT	CNTRL-REGISTER	0x60002	Spectrum config limit
Spectrum_0_CONFIG_REBIN	CNTRL-REGISTER	0x60003	Spectrum config rebin
Spectrum_0_CONFIG_MIN	CNTRL-REGISTER	0x60004	Spectrum config min
Spectrum_0_CONFIG_MAX	CNTRL-REGISTER	0x60005	Spectrum config max
Spectrum_1	Spectrum	0x70000	Spectrum base address
Spectrum_1_STATUS	CNTRL-REGISTER	0x80000	Spectrum status
Spectrum_1_CONFIG	CNTRL-REGISTER	0x80001	Spectrum config
Spectrum_1_CONFIG_LIMIT	CNTRL-REGISTER	0x80002	Spectrum config limit
Spectrum_1_CONFIG_REBIN	CNTRL-REGISTER	0x80003	Spectrum config rebin
Spectrum_1_CONFIG_MIN	CNTRL-REGISTER	0x80004	Spectrum config min
Spectrum_1_CONFIG_MAX	CNTRL-REGISTER	0x80005	Spectrum config max
Spectrum_2	Spectrum	0x90000	Spectrum base address
Spectrum_2_STATUS	CNTRL-REGISTER	0xa0000	Spectrum status
Spectrum_2_CONFIG	CNTRL-REGISTER	0xa0001	Spectrum config
Spectrum_2_CONFIG_LIMIT	CNTRL-REGISTER	0xa0002	Spectrum config limit
Spectrum_2_CONFIG_REBIN	CNTRL-REGISTER	0xa0003	Spectrum config rebin
Spectrum_2_CONFIG_MIN	CNTRL-REGISTER	0xa0004	Spectrum config min
Spectrum_2_CONFIG_MAX	CNTRL-REGISTER	0xa0005	Spectrum config max
Spectrum_3	Spectrum	0xb0000	Spectrum base address
Spectrum_3_STATUS	CNTRL-REGISTER	0xc0000	Spectrum status
Spectrum_3_CONFIG	CNTRL-REGISTER	0xc0001	Spectrum config
Spectrum_3_CONFIG_LIMIT	CNTRL-REGISTER	0xc0002	Spectrum config limit
Spectrum_3_CONFIG_REBIN	CNTRL-REGISTER	0xc0003	Spectrum config rebin
Spectrum_3_CONFIG_MIN	CNTRL-REGISTER	0xc0004	Spectrum config min
Spectrum_3_CONFIG_MAX	CNTRL-REGISTER	0xc0005	Spectrum config max
Spectrum_4	Spectrum	0xd0000	Spectrum base address
Spectrum_4_STATUS	CNTRL-REGISTER	0xe0000	Spectrum status
Spectrum_4_CONFIG	CNTRL-REGISTER	0xe0001	Spectrum config
Spectrum_4_CONFIG_LIMIT	CNTRL-REGISTER	0xe0002	Spectrum config limit
Spectrum_4_CONFIG_REBIN	CNTRL-REGISTER	0xe0003	Spectrum config rebin
Spectrum_4_CONFIG_MIN	CNTRL-REGISTER	0xe0004	Spectrum config min
Spectrum_4_CONFIG_MAX	CNTRL-REGISTER	0xe0005	Spectrum config max

Spectrum_5	Spectrum	0xf0000	Spectrum base address
Spectrum_5_STATUS	CNTRL-REGISTER	0x100000	Spectrum status
Spectrum_5_CONFIG	CNTRL-REGISTER	0x100001	Spectrum config
Spectrum_5_CONFIG_LIMIT	CNTRL-REGISTER	0x100002	Spectrum config limit
Spectrum_5_CONFIG_REBIN	CNTRL-REGISTER	0x100003	Spectrum config rebin
Spectrum_5_CONFIG_MIN	CNTRL-REGISTER	0x100004	Spectrum config min
Spectrum_5_CONFIG_MAX	CNTRL-REGISTER	0x100005	Spectrum config max
Spectrum_6	Spectrum	0x110000	Spectrum base address
Spectrum_6_STATUS	CNTRL-REGISTER	0x120000	Spectrum status
Spectrum_6_CONFIG	CNTRL-REGISTER	0x120001	Spectrum config
Spectrum_6_CONFIG_LIMIT	CNTRL-REGISTER	0x120002	Spectrum config limit
Spectrum_6_CONFIG_REBIN	CNTRL-REGISTER	0x120003	Spectrum config rebin
Spectrum_6_CONFIG_MIN	CNTRL-REGISTER	0x120004	Spectrum config min
Spectrum_6_CONFIG_MAX	CNTRL-REGISTER	0x120005	Spectrum config max
Spectrum_7	Spectrum	0x130000	Spectrum base address
Spectrum_7_STATUS	CNTRL-REGISTER	0x140000	Spectrum status
Spectrum_7_CONFIG	CNTRL-REGISTER	0x140001	Spectrum config
Spectrum_7_CONFIG_LIMIT	CNTRL-REGISTER	0x140002	Spectrum config limit
Spectrum_7_CONFIG_REBIN	CNTRL-REGISTER	0x140003	Spectrum config rebin
Spectrum_7_CONFIG_MIN	CNTRL-REGISTER	0x140004	Spectrum config min
Spectrum_7_CONFIG_MAX	CNTRL-REGISTER	0x140005	Spectrum config max
Spectrum_8	Spectrum	0x150000	Spectrum base address
Spectrum_8_STATUS	CNTRL-REGISTER	0x160000	Spectrum status
Spectrum_8_CONFIG	CNTRL-REGISTER	0x160001	Spectrum config
Spectrum_8_CONFIG_LIMIT	CNTRL-REGISTER	0x160002	Spectrum config limit
Spectrum_8_CONFIG_REBIN	CNTRL-REGISTER	0x160003	Spectrum config rebin
Spectrum_8_CONFIG_MIN	CNTRL-REGISTER	0x160004	Spectrum config min
Spectrum_8_CONFIG_MAX	CNTRL-REGISTER	0x160005	Spectrum config max
Spectrum_9	Spectrum	0x170000	Spectrum base address
Spectrum_9_STATUS	CNTRL-REGISTER	0x180000	Spectrum status
Spectrum_9_CONFIG	CNTRL-REGISTER	0x180001	Spectrum config
Spectrum_9_CONFIG_LIMIT	CNTRL-REGISTER	0x180002	Spectrum config limit
Spectrum_9_CONFIG_REBIN	CNTRL-REGISTER	0x180003	Spectrum config rebin
Spectrum_9_CONFIG_MIN	CNTRL-REGISTER	0x180004	Spectrum config min
Spectrum_9_CONFIG_MAX	CNTRL-REGISTER	0x180005	Spectrum config max
Spectrum_10	Spectrum	0x190000	Spectrum base address
Spectrum_10_STATUS	CNTRL-REGISTER	0x1a0000	Spectrum status
Spectrum_10_CONFIG	CNTRL-REGISTER	0x1a0001	Spectrum config
Spectrum_10_CONFIG_LIMIT	CNTRL-REGISTER	0x1a0002	Spectrum config limit
Spectrum_10_CONFIG_REBIN	CNTRL-REGISTER	0x1a0003	Spectrum config rebin
Spectrum_10_CONFIG_MIN	CNTRL-REGISTER	0x1a0004	Spectrum config min
Spectrum_10_CONFIG_MAX	CNTRL-REGISTER	0x1a0005	Spectrum config max
Spectrum_11	Spectrum	0x1b0000	Spectrum base address
Spectrum_11_STATUS	CNTRL-REGISTER	0x1c0000	Spectrum status

Spectrum_11_CONFIG	CNTRL-REGISTER	0x1c0001	Spectrum config
Spectrum_11_CONFIG_LIMIT	CNTRL-REGISTER	0x1c0002	Spectrum config limit
Spectrum_11_CONFIG_REBIN	CNTRL-REGISTER	0x1c0003	Spectrum config rebin
Spectrum_11_CONFIG_MIN	CNTRL-REGISTER	0x1c0004	Spectrum config min
Spectrum_11_CONFIG_MAX	CNTRL-REGISTER	0x1c0005	Spectrum config max
Spectrum_12	Spectrum	0x1d0000	Spectrum base address
Spectrum_12_STATUS	CNTRL-REGISTER	0x1e0000	Spectrum status
Spectrum_12_CONFIG	CNTRL-REGISTER	0x1e0001	Spectrum config
Spectrum_12_CONFIG_LIMIT	CNTRL-REGISTER	0x1e0002	Spectrum config limit
Spectrum_12_CONFIG_REBIN	CNTRL-REGISTER	0x1e0003	Spectrum config rebin
Spectrum_12_CONFIG_MIN	CNTRL-REGISTER	0x1e0004	Spectrum config min
Spectrum_12_CONFIG_MAX	CNTRL-REGISTER	0x1e0005	Spectrum config max
Spectrum_13	Spectrum	0x1f0000	Spectrum base address
Spectrum_13_STATUS	CNTRL-REGISTER	0x200000	Spectrum status
Spectrum_13_CONFIG	CNTRL-REGISTER	0x200001	Spectrum config
Spectrum_13_CONFIG_LIMIT	CNTRL-REGISTER	0x200002	Spectrum config limit
Spectrum_13_CONFIG_REBIN	CNTRL-REGISTER	0x200003	Spectrum config rebin
Spectrum_13_CONFIG_MIN	CNTRL-REGISTER	0x200004	Spectrum config min
Spectrum_13_CONFIG_MAX	CNTRL-REGISTER	0x200005	Spectrum config max
Spectrum_14	Spectrum	0x210000	Spectrum base address
Spectrum_14_STATUS	CNTRL-REGISTER	0x220000	Spectrum status
Spectrum_14_CONFIG	CNTRL-REGISTER	0x220001	Spectrum config
Spectrum_14_CONFIG_LIMIT	CNTRL-REGISTER	0x220002	Spectrum config limit
Spectrum_14_CONFIG_REBIN	CNTRL-REGISTER	0x220003	Spectrum config rebin
Spectrum_14_CONFIG_MIN	CNTRL-REGISTER	0x220004	Spectrum config min
Spectrum_14_CONFIG_MAX	CNTRL-REGISTER	0x220005	Spectrum config max
Spectrum_15	Spectrum	0x230000	Spectrum base address
Spectrum_15_STATUS	CNTRL-REGISTER	0x240000	Spectrum status
Spectrum_15_CONFIG	CNTRL-REGISTER	0x240001	Spectrum config
Spectrum_15_CONFIG_LIMIT	CNTRL-REGISTER	0x240002	Spectrum config limit
Spectrum_15_CONFIG_REBIN	CNTRL-REGISTER	0x240003	Spectrum config rebin
Spectrum_15_CONFIG_MIN	CNTRL-REGISTER	0x240004	Spectrum config min
Spectrum_15_CONFIG_MAX	CNTRL-REGISTER	0x240005	Spectrum config max
CP_0	CustomPacket	0x240007	CP_0 cp base address
CP_0_READ_STATUS	CNTRL-REGISTER	0x240008	CP read status
CP_0_READ_VALID_WORDS	CNTRL-REGISTER	0x240009	CP read valid words
CP_0_CONFIG	CNTRL-REGISTER	0x24000a	CP config

PCFG: register file for PSA core configuration (THIS PARAMETERS ARE ALL THE SAME FOR ALL CHANNELS)

RAWOFS: *offset raw applied to the input signal to remove the large DC offset.*

POL: *signal polarity*

- *0: positive*
- *1: negative*

TRG_SOURCE: *trigger datapath*

- *0: use shaper output for trigger*
- *1: use input signal for trigger*

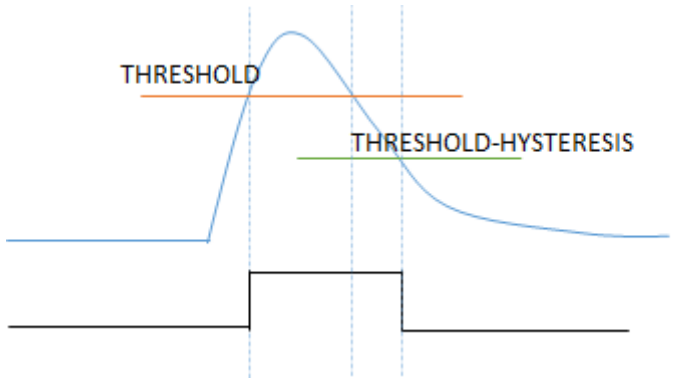
TRG_MODE: *internal trigger mode (external trigger is always enable)*

- *0: use the sum of the two side*
- *1: no internal trigger*
- *2: use the or between the two side*
- *3: use the and between the two side*

THRES: *Trigger threshold. value must be between 0 and 65535*

HIST: *Trigger hysteresis. This value is subtracted to the trigger on the falling edge of the signal in order to prevent from noise to double trigger on slow falling edge.*

Hysteresis must be always smaller than trigger



TRG_OUT_W: *Trigger output width (in samples). Width of the TRIGGER signal generated in output*

FLT_CFG: *Configuration of the filter stage. Suggest to start from 0x1F33*

- *bit 3..0: Attenuation section 1 (0 min, F max)*
- *bit 7..4: Attenuation section 2 (0 min, F max)*
- *bit 8 enable baseline correction of the input*
- *bit 9 enable pole zero compensation*
- *bit 10 enable first second-order gaussian filter section*
- *bit 11 second second-order gaussian filter section*
- *bit 12 baseline correction of the output*

Use the monitor signal AN_PROBEx to probe the internal filter stage signals and regulate the attenuation 1 and attenuation 2 in order to do not saturate any internal stage and do not loose resolution.

COEFF11

Filter coefficient
*int(exp(-2.71072*TS) * 16384 + 0.5)*

where
CLOCK_FREQUENCY := 125000000.0
tau is the decay time of the detector signal := 1e-6
*TS = (2 * pi) / (tau*CLOCK_FREQUENCY)*

COEFF12
Filter coefficient
*int(2 * exp(-1.35536*TS) * cos(0.327948*TS) * 16384 + 0.5)*

COEFF21
Filter coefficient
*int(exp(-2.36216*TS) * 16384 + 0.5)*

COEFF22
Filter coefficient
*int(2 * exp(-1.18108*TS)* cos(1.06037*TS) * 16384 + 0.5)*

PZCOEFF
Filter coefficient for pole zero
*int(32768 / (CLOCK_FREQUENCY*pz))*
where
pz is the decay time of the preamplifier := 0.1e-6

PRB_SEL

Analog monitor selector

- 0: signal with corrected offset and polarity
- 1: signal with corrected offset only
- 2: main filter output
- 3: sum of both side main filter output
- 4: peak detector output
- 5: peak detector output on sum signal
- 6: input baseline correction algorithm output
- 7: pz algorithm output
- 8: first filter algorithm output (divided by 2)
- 9: second filter algorithm output (divided by 2)
- A: output baseline algorithm output (divided by 2)

T0SW: T0 Software, set to 1 to reset timestamp

VETOSW: Software VETO, set to 1 to veto all triggers

EXTTRG_EN: Enable external trigger from Sync 2 (Configure lemo using the display)

RUN_EN: enable acquisition. Must be set to 1 to enable PSA core

TRG_MASK: *mask triger (one for each channel)*

- *0: both side enabled*
- *1: A side masked*
- *2: B side masked*
- *3: both masked*

FINE_OFFSET: *Subtract input offset on 12 bit signal. (one for each channel)*

Offset is subtracted before the polarity inversion

RATE_METER: Rate meter SciCompiler IP. (ie 16) registers from baseline with rate from each channel

OSCILLOSCOPE_X: Oscilloscope SciCompiler IP *(one for each channel).* Use SDK or read SciCompiler guide to download and decode data

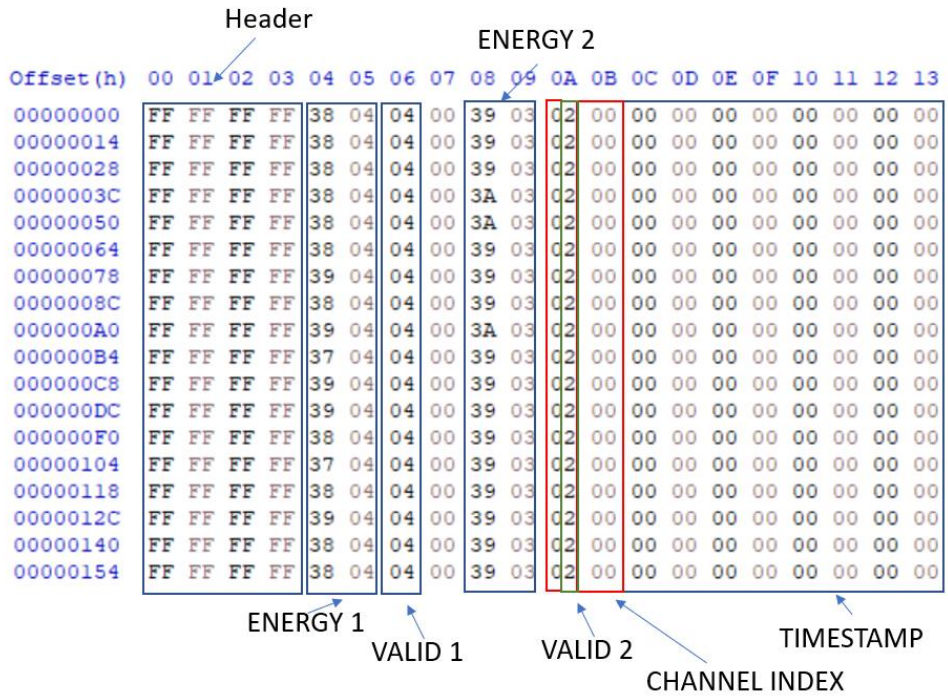
SPECTRUM_X: Spectrum SciCompiler IP *(one for each channel).* Use SDK or read SciCompiler guide to download and decode data

CP_X: Custom Packet List SciCompiler IP *(one for each channel).* Use SDK or read SciCompiler guide to download and decode data

CUSTOM PACKET OUTPUT DATA FORMART

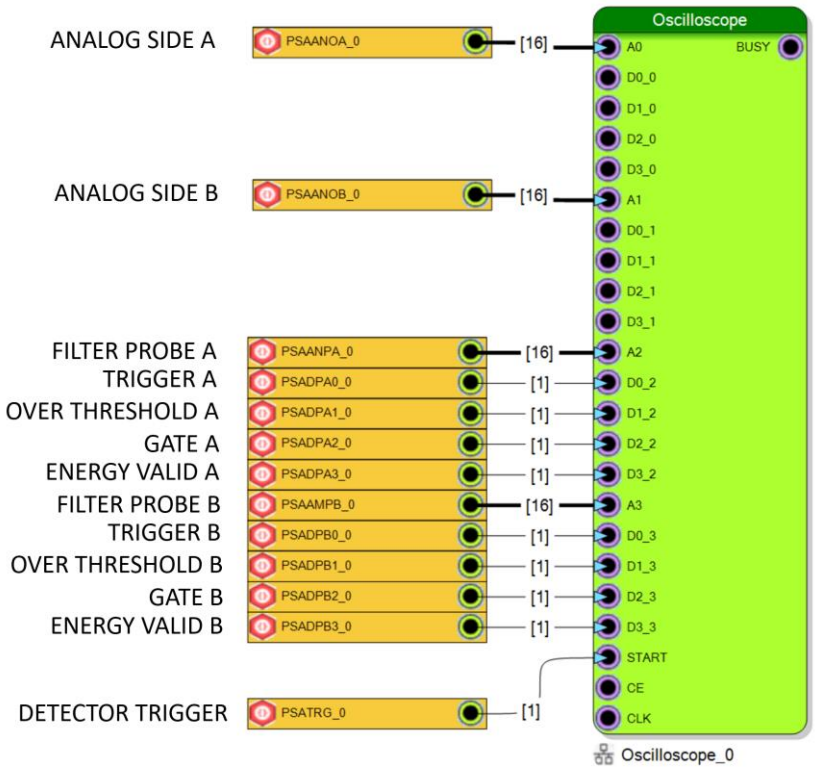
In order to make simpler the data decode the format transmitted from the data Custom Packet is slightly different from the format generated from the code. A 0XFFFFFFFF header is added to the PSA data output packet and few bit are set to flag valid events.

The following is a dump of a file stored in the same order of byte sent to the PC using the custom core and dumped on file by Resource Explorer



OSCILLOSCOPE BLOCK

An oscilloscope block for each detector is provided for monitor the analog and digital signals for each core

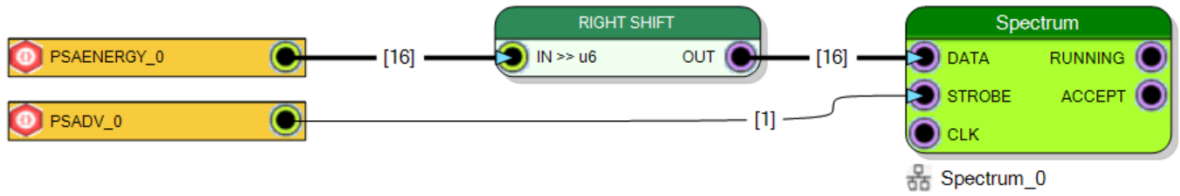


Set PRB_SEL to select the analog monitor point of the processing chain to be routed on FILTER PROBE.

SPECTRUM BLOCK

A spectrum block for each detector is provided to calculate the energy spectrum for each detector. The SUM output from PSA core is used to calculate the spectrum.

The spectrum works on 10 bits (1024 bins) indeed the SUM output (16 bit) is right shifted by six bits



REGISTER CONFIGURATION FOR DT1260 FIRMWARE EXAMPLE

The Example suppose a signal with a decay time of 1us

Table 0

Manual | Get All | Set All

Name	Description	Address	Format	Value Write	
ANALOG_OFFSET		FFFFFFF9	Decimal		Set
REGFILE_0_RUN		1009	Decimal	1	Set
REGFILE_0_POL		100A	Decimal	0	Set
REGFILE_0_OFFSET		100B	Decimal	0	Set
REGFILE_0_TRG_SOU...		100C	Decimal	0	Set
REGFILE_0_EN_COINC		100D	Decimal	0	Set
REGFILE_0_TRG_MODE		100E	Decimal	0	Set
REGFILE_0_TRG_THRS		100F	Decimal	1000	Set
REGFILE_0_TRG_HIST		1010	Decimal	100	Set
REGFILE_0_TRG_HOFF		1011	Decimal	0	Set
REGFILE_0_TRG_OUT...		1012	Decimal	10	Set
REGFILE_0_GATE_W		1013	Decimal	250	Set
REGFILE_0_TRG_MASK		1014	Decimal	0	Set
REGFILE_0_FLT_CFG		1015	Hexadecimal	1f37	Set
REGFILE_0_COEFF11		1016	Hexadecimal	30bc	Set
REGFILE_0_COEFF12		1017	Hexadecimal	6fa2	Set
REGFILE_0_COEFF21		1018	Hexadecimal	3279	Set
REGFILE_0_COEFF22		1019	Hexadecimal	7106	Set
REGFILE_0_COEFFPZ		101A	Hexadecimal	20c	Set
REGFILE_0_PRB_SEL		101B	Decimal	2	Set

COMPILE THE FIRMWARE ON R/DT5560

On version A of the R5560/DT5560 it is mandatory to set Area optimization in SciCompiler project and select in board configuration → board configuration → FPGA model → Z-7030-MINIMAL

REGISTER CONFIGURATION FOR R5560 FIRMWARE EXAMPLE

The Example suppose a signal with a decay time of 1us

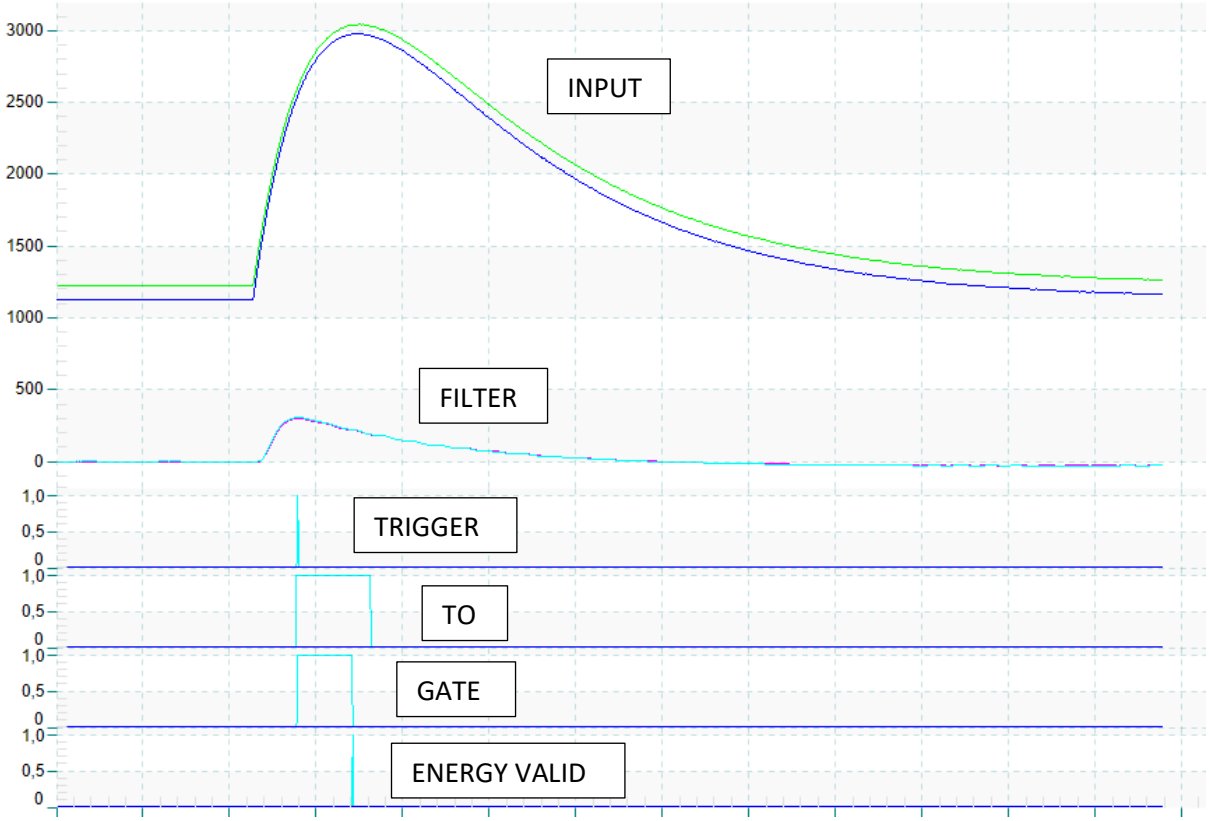
Name	Description	Address	Format	Value Write	
T0SW		11	Decimal	0	Set
VETOSW		12	Decimal	0	Set
EXTTRG_EN		13	Decimal	0	Set
RUN_EN		14	Decimal	1	Set
PCFG_RAWOFS		1	Hexadecimal	E4A8	Set
PCFG_POL		2	Decimal	0	Set
PCFG_TRG_SOURCE		3	Decimal	0	Set
PCFG_TRG_MODE		4	Decimal	0	Set
PCFG_THRS		5	Unsigned	10000	Set
PCFG_HIST		6	Decimal	1000	Set
PCFG_TRG_HOLD		7	Decimal	25	Set
PCFG_TRG_W		8	Decimal	10	Set
PCFG_GATE_W		9	Decimal	100	Set
PCFG_FLT_CFG		A	Hexadecimal	1f44	Set
PCFG_C11		B	Hexadecimal	37d9	Set
PCFG_C12		C	Hexadecimal	778e	Set
PCFG_C21		D	Hexadecimal	38d6	Set
PCFG_C22		E	Hexadecimal	7873	Set
PCFG_CPZ		F	Hexadecimal	106	Set
PCFG_PRB_SEL		10	Decimal	8	Set
TRG_MASK_CH0		20002	Decimal		Set
TRG_MASK_CH1		20003	Decimal		Set
TRG_MASK_CH2		20004	Decimal		Set
TRG_MASK_CH3		20005	Decimal		Set

It is not necessary to set all others parameters. The default 0 value is fine.

If the filter become unstable during the update of the values, toggle 0→1→0 the value of the RESET_CORE register in order to reload all parameters and resets accumulator inside the filters.

Expected signal

Set signed as data format



PYTHON CODE TO GENERATE REGISTERS

```
from cmath import pi
import math

CLOCK_FREQUENCY=125000000.0
attbit1 =3
attbit2 = 3
filtflip= 31

flag_attbit1 = (attbit1 & 0xf) + ((attbit2 & 0xf) << 4) + ((filtflip &
0xff) << 8)
tau=1e-6
pz = 1e-6
TS = (2 * pi) / (tau*CLOCK_FREQUENCY)
pzcoeff2 = (32768 / (CLOCK_FREQUENCY*pz))
coeff11_d = (math.exp(-2.71072*TS) * 16384 + 0.5)
coeff12_d = (2 * math.exp(-1.35536*TS)*math.cos(0.327948*TS) * 16384 + 0.5)

coeff21_d = (math.exp(-2.36216*TS) * 16384 + 0.5)
coeff22_d = (2 * math.exp(-1.18108*TS)* math.cos(1.06037*TS) * 16384 + 0.5)


print(hex(flag_attbit1))
print(hex(int(pzcoeff2)))
print(hex(int(coeff11_d)))
print(hex(int(coeff12_d)))
print(hex(int(coeff21_d)))
print(hex(int(coeff22_d)))
```